SCREENGUARD

AI-Powered Screen Distance Protection System

Complete Technical Project Documentation

Built by: Shivam Nanda

B.Tech CSE, GCE Gaya | MindForge Co-Founder | Learning Now by Shivam Nanda(yt channel)

Platform: Windows Desktop App (Python) + Android App (Kotlin) |

1. Project Overview & Problem Statement

ScreenGuard is a real-world software solution built to protect children's eyesight by automatically detecting when a child holds a phone or screen closer than 25 cm to their eyes, issuing a visible countdown warning, and locking the device if the child does not move back within 5 seconds.

The core problem this solves is digital myopia — a growing global health crisis where children who hold screens too close risk permanent short-sightedness. Unlike passive parental advisory apps, ScreenGuard acts in real-time, without requiring a parent to be physically present.

1.1 Key Features (Final Build)

Feature	Description
Real-Time Detection	Uses device camera + AI to measure face proximity continuously in the background
Dynamic Distance Math	Calculates proximity using face proportions relative to frame size (device-agnostic)
5-Second Countdown Warning	Visible warning card/overlay counts down live before any lock action
Screen Lock Trigger	Forces OS-level screen lock if user does not move back within 5 seconds
Multi-Face Support (Android)	Detects any face in frame — not just the first one found
Angle Compensation (Android)	Uses trigonometry (cosine) to catch side-profile views like a child lying on a bed
True Background Operation (Android)	Works invisibly while child uses any app — YouTube, games, browser, etc.
System Tray Operation (Windows)	Runs silently in Windows system tray — no visible terminal window; quit via right-click
Battery & Power Protection (Android)	Pauses camera when screen is off to prevent phantom loops and drain
Zero Privacy Risk	No internet permission on either version. All video frames processed and destroyed locally.
Standalone Executable (Windows)	Compiled with PyInstaller into a single .exe — no Python installation needed to run
Shareable APK (Android)	Compiled into .apk file — installable on any Android phone without developer tools

1.2 Two Versions Built

	Version 1 — Windows Desktop	Version 2 — Android App
Language	Python 3.x	Kotlin
AI Library	OpenCV Haar Cascade Classifier	Google ML Kit Face Detection API
Detection Method	Face bounding box width vs calibration constant	Normalized face/frame ratio with angle compensation
Screen Lock	os.system() — rundll32 / pmset / xdg-screensaver	AccessibilityService GLOBAL_ACTION_LOCK_SCREEN
Background Mode	System Tray icon (pystray) + background thread	AccessibilityService + WindowManager overlay
Distribution	PyInstaller .exe (double-click to run)	Android .apk file (share and install)
Runs On	Windows / Mac / Linux with webcam	Any Android phone

2. Version 1 — Windows Desktop Application (Python)

2.1 Files Used

File Name	Type	Purpose
screen_guard.py	Python Script	The single main file. Contains: system tray setup, background threading, webcam capture, face detection, distance calculation, warning overlay, and screen lock logic. Everything in one file.
haarcascade_frontalface_default.xml	ML Model File	Pre-trained AI face detection model bundled inside OpenCV. Auto-downloaded from official OpenCV GitHub if not found locally (self-healing mechanism in final build).
screen_guard.exe (output)	Compiled Executable	The final distributable file generated by PyInstaller. Single file, no Python required. Placed inside the /dist/ folder after compilation.
screen_guard.spec (auto-generated)	PyInstaller Config	Auto-generated by PyInstaller during compilation. Describes how the .exe was built. Not manually edited.

TOTAL USER-WRITTEN FILES: 1 (screen_guard.py). All other files are either auto-generated by PyInstaller or bundled from OpenCV's package.

2.2 Libraries & Tools

Library / Tool	Role in This Project
cv2 (OpenCV)	Opens the webcam via VideoCapture(0), reads frames, detects faces using the Haar Cascade model, draws visual overlays (rectangles, text) on the live feed, displays the monitoring window
pystray	Creates and manages the Windows System Tray icon — the small icon in the bottom-right corner near the clock. Provides a right-click menu with a 'Quit' option.
PIL / Pillow	Required by pystray to render the tray icon image. Used to programmatically draw the icon graphic (dark background + red dot) in code.
threading	Python's built-in multi-threading module. Runs the OpenCV camera loop on a separate background thread so it does not block the system tray UI thread.
math	Standard Python library — calculates Euclidean distance or handles distance formula math for pixel-to-cm conversion
time	Tracks the 5-second countdown by recording when proximity was first detected and comparing to current time
os	Executes the OS-level screen lock terminal command from inside Python

Library / Tool	Role in This Project
platform	Detects the current operating system (Windows / Mac / Linux) so the correct lock command is used
urllib / requests (self-heal)	Downloads the haarcascade model XML file from OpenCV's GitHub if it cannot be found locally — prevents silent failures in compiled .exe builds
PyInstaller (build tool)	Packages the entire Python script + all dependencies into a single standalone .exe file. Not imported in code — run as a terminal command: python -m PyInstaller --onefile --noconsole screen_guard.py

2.3 Architecture — How the Desktop App Runs

A. The Two-Thread Architecture (Why It Matters)

A key problem with running OpenCV in a basic loop is that it would block all other code from running simultaneously. The app would either run the camera OR manage the tray icon — not both at once.

The solution is Python's threading module. The app is split into two independent execution paths running at the same time:

Thread	What It Does
Main Thread	Runs pystray.Icon.run() — keeps the system tray icon alive, listens for right-click menu selections, and manages the app's lifecycle. This thread must not be blocked.
Background Thread (daemon=True)	Runs the continuous OpenCV camera loop — captures frames, detects faces, calculates distance, shows warnings, and triggers the screen lock. Runs independently of the UI thread.

daemon=True is critical — it means when the main thread stops (user clicks Quit), the background camera thread automatically dies too. Without this, the background thread would keep running forever even after the app appears closed.

B. Global Flag for Safe Thread Shutdown

A global boolean variable (is_running = True) acts as the communication channel between threads. When the user clicks 'Quit Application' from the tray menu:

- The main thread's on_exit() function sets is_running = False
- The background thread checks this flag in every loop cycle and exits its while loop cleanly
- The camera (cap.release()) is properly closed before the thread terminates

This is the standard thread-safe shutdown pattern in Python — avoids force-killing threads which can leave the camera hardware in a locked state.

C. Face Detection — Haar Cascade Classifier

OpenCV includes a pre-trained machine learning model called a Haar Cascade Classifier. This model was trained by researchers who showed it thousands of face images and non-face images. It learned to identify patterns — edges, contrast zones, dark eye sockets against lighter foreheads — that are mathematically specific to human faces.

When the app runs, it loads this model and passes every webcam frame through it. The model outputs a bounding box — a rectangle marking where the face is (x, y, width, height in pixels).

D. Distance Estimation — The Focal Length Formula

Distance (cm) = Calibration Constant / Pixel Width of Face Bounding Box

Calibration Constant = Focal Length x Real Face Width = ~600 x ~14 cm = ~8,400

Variable	Explanation
Focal Length (~600)	Approximate focal length of a standard 640x480 webcam in pixel units — a hardware constant
Real Face Width (~14 cm)	Average human adult face width in centimeters — a biological constant
Pixel Width of Face	How wide the detected bounding box is in the current frame — changes as person moves
Calibration Adjustment	Because every webcam lens has different focal properties, the constant 8,400 is tuned by sitting at exactly 25 cm and adjusting the number until the displayed distance matches the physical ruler measurement

CRITICAL DIFFERENCE vs Android: The desktop version uses a fixed pixel formula because it runs on a fixed laptop webcam. The Android version uses a normalized ratio (face/frame %) because it must work across many different phone cameras with different resolutions.

E. The Self-Healing Model Loader

A major bug discovered during testing: when PyInstaller packages the app into a .exe file, it cannot always locate OpenCV's bundled model file (haarcascade_frontalface_default.xml) because the file path changes inside the compiled executable environment.

The fix: the final script includes a self-healing model loader. At startup, it checks if the model file exists in the expected location. If it is missing (which happens inside .exe packaging), it automatically downloads the official, correct version directly from OpenCV's GitHub repository and stores it in a local cache folder. This makes the .exe completely self-sufficient on first launch.

F. The Proximity Threshold — Calibrated for Laptops

Laptop webcams have a wider field-of-view than mobile phone cameras. A face ratio threshold calibrated for phones would be too sensitive on a laptop — the screen would lock constantly at normal working distances. The threshold was lowered to 0.28 (28% of frame) in the final desktop build, corresponding to approximately 25 cm at typical laptop webcam positions.

G. System Lock — Cross-Platform

OS	Lock Command
Windows	rundll32.exe user32.dll,LockWorkStation — calls a built-in Windows system DLL function directly
macOS	pmset displaysleepnow — puts the display to sleep immediately
Linux	xdg-screensaver lock — standard cross-desktop screen locker command

2.4 Compilation — How to Make a Shareable .exe

PyInstaller is a build tool that bundles a Python script + all its library dependencies into one standalone executable. The user running the .exe does NOT need Python installed.

1. Install PyInstaller: pip install pyinstaller
2. Kill any running instance: taskkill /f /im screen_guard.exe
3. Delete old build folders (build/ and dist/) to avoid cache conflicts
4. Run compilation: python -m PyInstaller --clean --onefile --noconsole screen_guard.py
5. Find finished file inside the auto-created /dist/ folder as screen_guard.exe

PyInstaller Flag	What It Does
--onefile	Bundles everything (Python runtime + all libraries) into a single .exe file instead of a folder
--noconsole	Hides the black terminal/command prompt window when the .exe is launched — pure invisible background operation
--clean	Clears PyInstaller's cache before building — prevents stale cached files from causing permission errors
-m flag	Running as 'python -m PyInstaller' instead of just 'pyinstaller' bypasses Windows PATH environment bugs where the pyinstaller command is not recognized

2.5 Windows Iteration History

ITERATION W1 — Basic Script (Terminal-Only)

Status	First working build — face detection and screen lock in a simple while loop
How to Run	Open terminal, navigate to folder, type: python screen_guard.py
Problem	Closing the terminal window kills the app. No background operation. Not user-friendly.
Lock Method	os.system() with platform-specific commands
Libraries Used	cv2, math, time, os, platform

ITERATION W2 — MediaPipe Eye-Tracking Attempt (Failed)

Attempt	Tried using MediaPipe FaceMesh for more precise eye-corner tracking instead of face bounding box

Math Used	Interpupillary Distance (IPD): distance between inner eye corners converted to cm via focal length formula
Problem	MediaPipe completely removed mp.solutions API in newer versions. Python 3.13 also has deep structural conflicts with MediaPipe on Windows.
Resolution	Rolled back to OpenCV's built-in Haar Cascade — completely stable, zero external dependencies, immune to version conflicts

ITERATION W3 — Added System Tray + Background Threading

Problem Solved	App needed to run in background like a real Windows application, not a terminal script
Libraries Added	pystray (system tray icon), pillow/PIL (icon image drawing), threading (parallel execution)
Architecture Change	Split app into two threads: Main Thread runs pystray (tray icon + menu), Background Thread runs OpenCV camera loop
Control Mechanism	Global boolean flag is_running — shared between threads to safely signal shutdown
Icon Design	Programmatic PIL-drawn icon: dark background + red ellipse. Created in code, no external image file needed
Menu	Right-click shows: 'ScreenGuard Active' (disabled status label) and 'Quit Application' option
Result	App now runs invisibly in Windows taskbar tray. No terminal window visible.

ITERATION W4 — PyInstaller Compilation (First Attempt — Failed)

Goal	Package into a standalone .exe so anyone can run it without Python installed
Command Used	python -m PyInstaller --onefile --noconsole screen_guard.py
Problem 1	pystray.Item() naming mismatch — correct API is pystray.MenuItem(). App crashed on launch with no error window (--noconsole hides crash messages).
Problem 2	haarcascade_frontalface_default.xml model file could not be located inside the compiled .exe environment — face detection silently failed
Problem 3	PermissionError WinError 5: old screen_guard.exe was still running in background, locking the file so PyInstaller could not overwrite it
Fix for WinError 5	Run: taskkill /f /im screen_guard.exe, then manually delete build/ and dist/ folders, then re-run PyInstaller with --clean flag

ITERATION W5 — Final Build (All Issues Fixed)

Fix 1: API Name	Changed pystray.Item() to pystray.MenuItem() — the correct class name in the current pystray version
Fix 2: Self-Healing Model	Added automatic model downloader: at startup, checks if haarcascade XML exists; if missing, downloads from official OpenCV GitHub repo and caches locally. Works perfectly inside .exe environment.
Fix 3: Threshold Calibration	Lowered proximity threshold from 0.42 (phone-optimized) to 0.28 (laptop webcam-optimized) so detection triggers correctly at real-world laptop distances
Final Compilation Command	python -m PyInstaller --clean --onefile --noconsole screen_guard.py
Result	Double-click .exe launches silently, tray icon appears, camera detects face proximity in background, warning appears on screen, locks PC after 5 seconds if not addressed
Distribution	Share single screen_guard.exe file — works on any Windows PC without Python, libraries, or any setup

3. Version 2 — Android Application (Kotlin)

3.1 Files Used

File Name	Type	Purpose
AndroidManifest.xml	Config XML	Declares all permissions and registers the background service with the Android OS
build.gradle.kts (app level)	Build Script	Declares CameraX and ML Kit dependencies for Android Studio to download
accessibility_config.xml	Config XML	Defines accessibility service properties so Android registers it under Settings > Accessibility
ScreenGuardService.kt	Kotlin Source	Main brain: background camera loop, ML Kit detection, proximity calculation, warning banner UI, countdown, screen lock
MainActivity.kt	Kotlin Source	App front screen: requests permissions, shows 'Open Accessibility Settings' button
ic_launcher (in /res/)	PNG Assets	App icon — auto-resized by Android Studio's Image Asset Studio into all required screen density sizes

TOTAL USER-WRITTEN FILES: 5 (2 Kotlin + 2 XML + 1 build script). Icon files are auto-generated by Android Studio.

3.2 Libraries & Tools

Library / Tool	Role in This Project
Android Studio (IDE)	Development environment — writes, builds, debugs, and deploys the app to phone
Kotlin Language	Modern Android programming language — safer and more concise than Java
CameraX (androidx.camera)	Google's camera API. Handles camera permissions, lifecycle management, and image frame streaming
Google ML Kit Face Detection	On-device neural net model. Detects faces, returns bounding boxes and Euler angles (yaw, pitch, roll)
AccessibilityService API	Grants screen lock capability and background camera access via elevated system permissions
WindowManager API	Creates floating overlay windows on top of all other apps — hosts the warning banner
PowerManager API	Queries whether screen is awake (isInteractive) — used to pause camera when screen is off
GradientDrawable	Programmatic UI styling — creates rounded corners (cornerRadius=30f) and flat modern red (#DC2626) on the warning card

Library / Tool	Role in This Project
Handler + Looper	Safely dispatches UI updates (warning banner text changes) from background camera thread to main UI thread
LifecycleRegistry	Links the background service to Android's lifecycle system so CameraX knows when to start/stop the camera

3.3 Core Concepts Used (Android)

A. Why AccessibilityService (Not a Regular Service)

Android OS blocks background apps from accessing the camera and from locking the screen — these would be severe privacy/security violations if any random app could do them. The Accessibility Service API is the only legal pathway to both capabilities without root access. It was originally designed for assistive technology (screen readers, motor impairment tools) and is therefore granted protected system-level privileges.

B. The Invisible Window Trick

Android only allows camera access when there is a visible component on screen. ScreenGuard uses a WindowManager.LayoutParams.TYPE_ACCESSIBILITY_OVERLAY to create a floating view layer. This satisfies Android's 'visible component' requirement while being the same overlay that displays the warning banner — the overlay serves double duty.

C. Proximity Calculation — Normalized Ratio (Device-Agnostic)

Proximity Ratio = Face Bounding Box Width (px) / Camera Frame Width (px)

Height Ratio = Face Bounding Box Height (px) / Camera Frame Height (px)

Unlike the desktop version's fixed pixel formula, this ratio works identically across all Android phones regardless of camera resolution (720p, 1080p, 4K) because it measures the face as a fraction of the total frame — not as an absolute pixel count.

D. Trigonometric Angle Compensation

Compression Factor = cos(Yaw Angle in Radians) x cos(Pitch Angle in Radians)

Adaptive Threshold = max(0.34, 0.40 x Compression Factor)

When someone views a phone at an angle (lying on bed, side view), their face appears narrower in the camera due to perspective projection. cos(angle) gives the exact ratio of how much width is preserved. Multiplying the base threshold by this factor automatically lowers sensitivity for angled views — catching side profiles that would otherwise be missed.

E. Grace Period + Battery Protection

- Grace Period (1.2s): Countdown only resets if face has been absent for more than 1,200ms — prevents a single blurry frame from resetting the full 5-second timer
- Battery Protection: if (!powerManager.isInteractive) — exits camera analysis immediately when screen is off, preventing phantom lock loops and unnecessary processing

4. Android App — Full Iteration History

ITERATION A1 — First Working Version

Detection	ML Kit with fixed pixel threshold: face bounding box width > 260 pixels = too close
Overlay	1x1 invisible FrameLayout — functional but zero user feedback
Warning	None — screen locks silently after 5 seconds
Problem	Blank white screen on launch. No UI. User cannot control or verify app is working.

ITERATION A2 — Settings UI + Visible Warning Banner

Added	Jetpack Compose UI in MainActivity — proper screen with 'Open Accessibility Settings' button
Added	Visible red TextView banner with live countdown text (View.VISIBLE / View.GONE toggle)
New Concepts	Jetpack Compose setContent{}, Handler(Looper.getMainLooper()).post{} for thread-safe UI updates

ITERATION A3 — Multi-Face Detection

Problem	Only faces[0] (first detected face) was monitored — second child close to screen could be missed
Fix	Changed to faces.any { it.boundingBox.width() > 260 } — checks all faces simultaneously
Concept	Kotlin .any{} higher-order function — returns true if any element in collection matches condition

ITERATION A4 — Accuracy Upgrade (3 Improvements)

Fix 1	Replaced fixed 260px threshold with normalized ratio (face/frame > 0.42) — works across all phone resolutions
Fix 2	Added height ratio alongside width — triggers if EITHER width OR height exceeds threshold (catches tilt angles)
Fix 3	Switched ML Kit from PERFORMANCE_MODE_FAST to PERFORMANCE_MODE_ACCURATE — better low-light detection
Fix 4	Added 1,200ms grace period — prevents single dropped frames from resetting the countdown

ITERATION A5 — UI Redesign (Modern Flat Card)

Problem	Plain red rectangle looked rough and outdated on modern phones
Fix	Added GradientDrawable with cornerRadius=30f, #DC2626 flat red, 45dp side margins for floating card look
Concept	GradientDrawable = Android's equivalent of CSS border-radius + background-color

ITERATION A6 — Trigonometric Angle Compensation

Problem	Phone flat on bed + child at side angle: face appears narrow in camera, ratio drops below threshold even at dangerous distance. Detection fails.
Also	Toddlers (2-year-olds) have smaller faces — old threshold missed them even extremely close
Fix	Applied cos(headEulerAngleY) x cos(headEulerAngleX) as compression factor. Multiplied base 0.40 threshold by this factor to get adaptive threshold. Floor of 0.34 prevents over-sensitivity.
Math	cos() from kotlin.math, Math.toRadians(), maxOf() for floor enforcement

ITERATION A7 — Battery Protection & Loop Fix (Final Build)

Problem	After screen locks, camera kept running in dark frames and sometimes re-triggered the lock immediately on unlock
Fix	PowerManager check at start of every frame: if (!powerManager.isInteractive) → close frame, reset state, return immediately
Concept	powerManager.isInteractive = Android API returning true only when screen is awake and actively in use
Result	Zero phantom loops, zero battery drain during lock, clean behavior on every unlock

5. Permission Architecture

5.1 Windows (No Special Permissions Required)

The Windows desktop version requires zero special OS permissions. Python scripts run with standard user-level privileges. The os.system() screen lock command uses rundll32.exe which is a standard Windows utility accessible to all users without elevation. The camera is accessed via OpenCV's VideoCapture() which prompts the user automatically on first run.

5.2 Android Permissions

Permission	Why Needed
CAMERA	Required to access the front-facing camera stream for face detection
SYSTEM_ALERT_WINDOW	Required to display the floating warning overlay on top of other apps
FOREGROUND_SERVICE	Declares a long-running background process so Android does not kill it to save memory
FOREGROUND_SERVICE_CAMERA	Android 14+ specific — must explicitly declare camera use inside a foreground service
BIND_ACCESSIBILITY_SERVICE	Registers the service as Accessibility Service, granting screen lock capability via GLOBAL_ACTION_LOCK_SCREEN

PRIVACY GUARANTEE: The Android app has NO INTERNET permission. Android's kernel physically blocks all network access for any permission not declared in AndroidManifest.xml — it is impossible for the app to transmit any camera data regardless of what the code does.

6. Complete Tech Stack — Side by Side

Component	Windows (Python)	Android (Kotlin)
Language	Python 3.x	Kotlin
IDE / Editor	Any text editor + Terminal	Android Studio
Camera Access	cv2.VideoCapture(0)	CameraX ImageAnalysis
Face Detection	OpenCV Haar Cascade Classifier (traditional ML)	Google ML Kit Neural Network (deep learning)
Detection Output	Bounding box (x, y, w, h) in pixels only	Bounding box + Euler angles (yaw, pitch, roll)
Distance Calculation	Focal Length Formula: ~8400 / pixel_width of face	Normalized ratio: face_width / frame_width (%)
Threshold Type	Fixed constant (calibrated per webcam)	Adaptive (0.34–0.40, angle-dependent via cos())
Warning Display	cv2 overlay text on live webcam window	GradientDrawable styled floating TextView card
Screen Lock	os.system() terminal command	performGlobalAction(GLOBAL_ACTION_LOCK_SCREEN)
Background Mode	threading.Thread (pystray main + OpenCV worker)	AccessibilityService (system-level protected process)
User Interface	System Tray icon + right-click menu	Jetpack Compose screen + accessibility settings button
Multi-Face	No (first face only)	Yes (faces.any{} checks all faces)
Angle Handling	No — frontal faces only reliable	Yes — cosine-based Euler angle compensation
Battery Management	Not applicable	PowerManager.isInteractive check per frame
Distribution	PyInstaller .exe — double-click, no setup	Android .apk — share and install on any phone
Privacy	No network calls, local only	No INTERNET permission; on-device AI; frames destroyed instantly

7. Key Concepts Glossary

Term / Concept	Plain-Language Explanation
Haar Cascade Classifier	Traditional ML face detector trained by showing thousands of face/non-face images. Fast but works mainly with frontal faces. Used in Windows version.
ML Kit Neural Network	Google's deep learning face detector. More accurate at angles and in low light. Provides Euler angles. Used in Android version.
Bounding Box	Rectangle the AI draws around a detected face: top-left (x, y), width, and height in pixels.
Focal Length Formula	Physics formula: Distance = (Focal_Length x Real_Size) / Pixel_Size. Converts pixel measurements to real-world centimeters.
Normalized Ratio	Face measurement as a fraction of total frame (e.g., 40%). Works the same on all screen resolutions — device-agnostic.
Euler Angles (Yaw/Pitch/Roll)	Three angles describing 3D head orientation. Yaw = left/right rotation. Pitch = up/down tilt. Roll = head tilt sideways.
Perspective Distortion	When viewed at an angle, objects appear narrower than actual. cos(60°) = 0.5 means face appears half its real width at 60-degree side view.
Threading (Python)	Running two code paths simultaneously. Main thread handles UI (tray icon). Background thread runs camera loop. Both execute at the same time.
daemon=True	Makes a background thread automatically die when the main thread exits — prevents zombie processes running after app is closed.
Global Flag (is_running)	A shared boolean variable used as a safe signal between threads. Setting it False tells all threads to stop gracefully.
pystray	Python library for creating Windows/Mac/Linux system tray icons with right-click menus.
PyInstaller	Packages Python scripts + all dependencies into a single standalone executable (.exe on Windows) that runs without Python installed.
Self-Healing Loader	Code that automatically detects a missing dependency (model file) and downloads it at runtime instead of crashing.
AccessibilityService (Android)	System-level API designed for assistive apps. Grants special permissions including screen lock control and background camera access.
WindowManager Overlay	Floating view layer rendered on top of all apps. Used for the warning banner that appears while the child uses YouTube or games.
isInteractive (Android)	PowerManager method returning true when screen is awake and in active use. Used to pause camera when screen turns off.
GradientDrawable (Android)	Programmatic drawing API — like CSS in Android. Sets rounded corners (cornerRadius), fill colors, and borders on UI elements in code.

Term / Concept	Plain-Language Explanation
Grace Period	A time buffer (1.2s) before resetting countdown when detection is lost. Prevents a single blurry frame from resetting the full 5-second timer.
State Machine	Programming pattern where app is always in one of fixed states (No Face / Safe / Too Close) and moves between them on specific conditions.
APK File	Android installer package — like .exe for Windows. Any Android phone can install it directly without Google Play Store or developer tools.